

FINAL EXAM PORTFOLIO

ITAI-4370: AI/5G Communications & ORAN Networks

Judith Barrios

Instructor: Tawanda Chiyangwa

Houston Community College

July 9, 2026

Introduction

This portfolio covers my work in ITAI-4370: AI/5G Communications & ORAN Networks, taught by Professor Tawanda Chiyangwa. The assignments and labs I've completed so far span telecom basics, 5G core architecture, AI in radio access networks, network traffic forecasting, running AI models on edge devices, and the ethics of using AI in real networks. Below, each module lists what I did, what came out of it, and what I actually learned.

All the lab code was written in Python, using libraries like NumPy, pandas, scikit-learn, statsmodels, and TensorFlow/Keras. Every assignment and lab here is my own work. Screenshots of my course module list are below for reference.

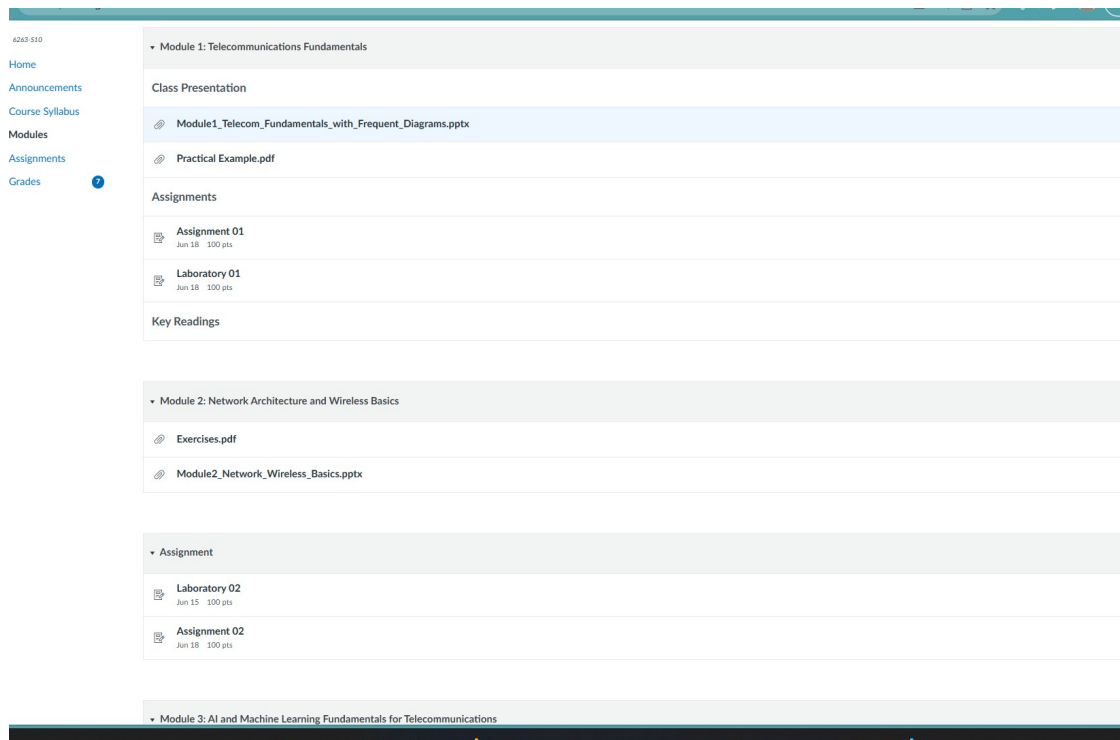


Figure 0.1: Course Modules 1–2, from the class website

▼ Module 3: AI and Machine Learning Fundamentals for Telecommunications
 Module3_AI_ML_Telecom.pptx
 Exercise 3_1 to 3_3.pdf
 Assignment 03 Jun 29 100 pts
 Laboratory 03 Jun 25 100 pts
▼ Module 4: 5G Network Architecture and AI Integration
 AI-Driven Load Balancing in Paravirtualized 5G Networks.pdf
 Integrating_Artificial_Intelligence_Internet_of_Things_and_5G_for_Next-Generation_Smartgrid_A_Survey_of_Trends_Challenges_and_Prospect.pdf
 Module_4_5G_AI_Integration.pptx
 Hands On Exercise.pdf
▼ Module 5: Open RAN (ORAN) and AI-Driven Networks
 Module5_OpenRAN_AI.pptx
 Advancements in Mobile Edge Computing.pdf
 From DeepSense to Open RAN.pdf
▼ Module 6: AI for Network Optimization and Management

Figure 0.2: Course Modules 3–6, from the class website

Module 1: Telecommunications Fundamentals

Activities: Assignment 01, Telecommunications Fundamentals | Laboratory 01, Communication System Diagram & BPSK Signal Flow

Assignment 01: Telecommunications Fundamentals

This one was pretty much an overview of how communication systems work at a high level. I wrote about what a telecommunications system actually is, walked through the transmitter-channel-receiver setup, and compared analog signals to digital ones. The last part had me compare different network layouts (star, mesh, ring, bus) and think through why some hold up better than others when something breaks.

Key takeaway

The topology comparison was the part that made the most sense to me. Star, mesh, ring, and bus all trade cost for reliability in different ways. A mesh network fixes the single-point-of-failure problem a star network has, but only by adding more cables and more cost. That same tradeoff comes back later in the course when we get to Open RAN design.

Laboratory 01: Communication System Diagram & BPSK Signal Flow

For this lab I first drew a basic communication system as a block diagram: source, modulator, channel, demodulator, receiver. Then I simulated that same flow in Python using BPSK modulation. The code generates 100,000 random bits per noise level, turns them into BPSK symbols, adds Gaussian noise to simulate the channel, decodes them with a simple threshold rule, and counts how many bits came out wrong. I compared that simulated error rate against the known theoretical curve across a noise range of 0 to 14 dB.

```
sigma = np.sqrt(1 / (2 * snr))
noise = sigma * np.random.randn(N)
received = symbols + noise
detected = (received > 0).astype(int)
errors = np.sum(bits != detected)
```

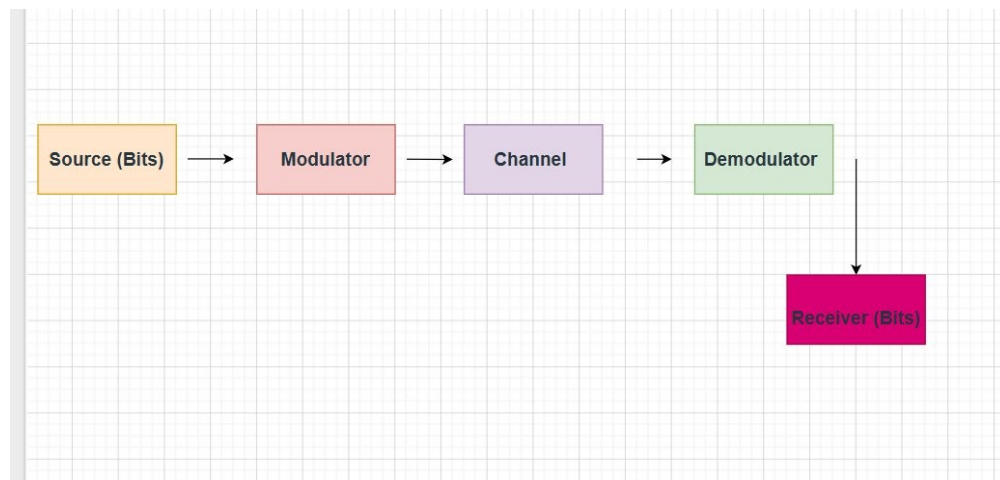


Figure 1.1: Simulated vs. theoretical bit error rate for a BPSK link, 0–14 dB

My simulated results landed right on top of the theoretical curve, which told me the simulation was working correctly. As the signal-to-noise ratio went up, the error rate dropped fast. By around 10 dB there were almost no errors left.

What I learned

This lab made the block diagram feel real instead of just a picture in a slide deck. Seeing my simulated points match the theoretical curve was a good check that I actually understood how noise messes with a digital signal, and why SNR matters so much in any radio link.

Module 2: Network Architecture and Wireless Basics

Activities: Laboratory 02, Wireshark Capture & RF Propagation Modeling | Assignment 02, 5G Core Network & Architecture

Laboratory 02, Part 1: Network Protocol Analysis with Wireshark

I installed Wireshark, captured about a minute of live traffic on my Wi-Fi (1,287 packets total), and used display filters to isolate IP, TCP, and HTTP traffic. Then I looked at the packet headers to see source/destination IPs, protocols, and port numbers.

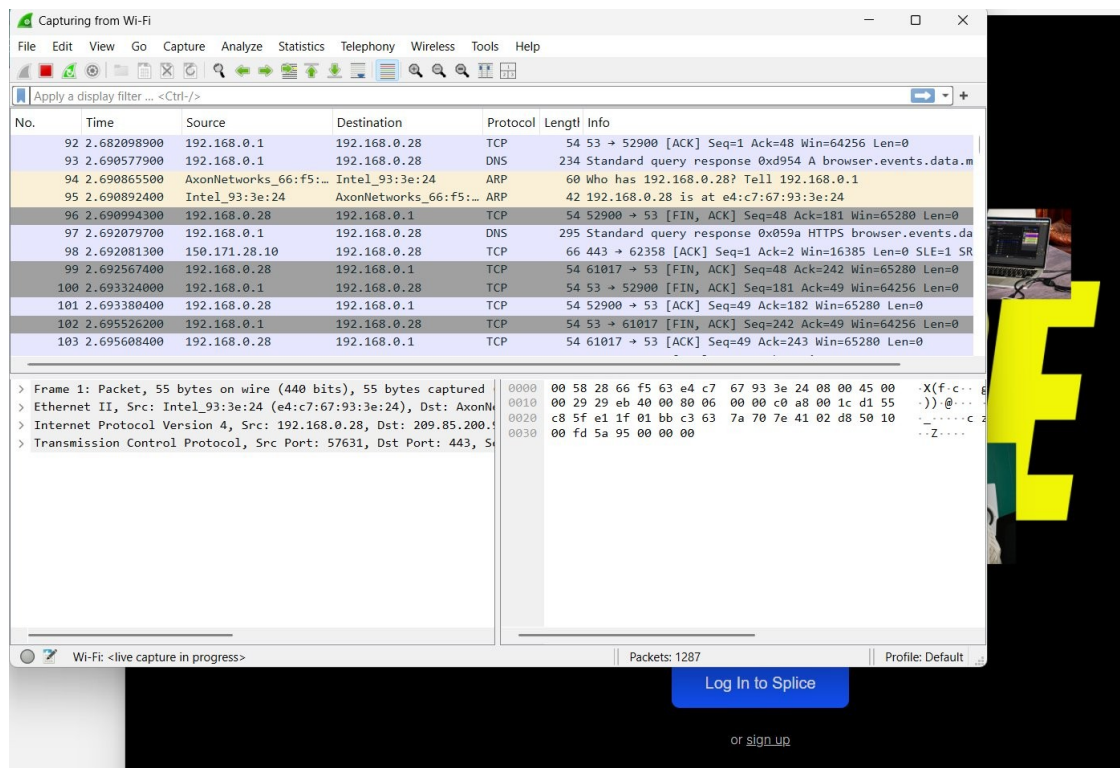


Figure 2.1: Live Wireshark capture on my Wi-Fi interface, 1,287 packets

My laptop (192.168.0.28) was talking to my router (192.168.0.1) and an outside server. The capture showed TCP packets opening and closing connections, DNS lookups resolving website names before each connection, and ARP traffic where devices on my network find each other's addresses. Every site I visited used port 443, meaning HTTPS. Encrypted traffic is basically the default now.

Laboratory 02, Part 2: RF Propagation Modeling with Python

For the second part, I modeled Free-Space Path Loss (FSPL) for a 2.4 GHz Wi-Fi signal at distances from 0.1 to 20 km, using the formula $FSPL \text{ (dB)} = 20 \cdot \log_{10}(d) + 20 \cdot \log_{10}(f) + 32.44$.

```
frequency = 2400 # MHz
distances = np.linspace(0.1, 20, 100) # km
fspl = 20*np.log10(distances) + 20*np.log10(frequency) + 32.44
```

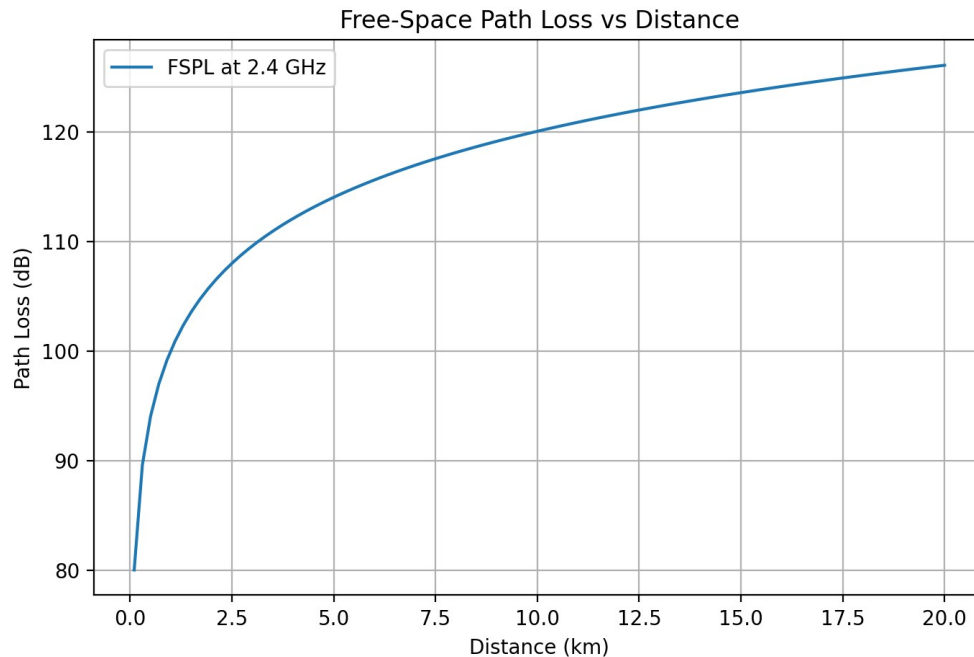


Figure 2.2: Free-space path loss vs. distance at 2.4 GHz

Distance (km)	FSPL (dB)
0.10	80.04
2.11	106.53
6.13	115.79
10.15	120.17
18.19	125.24

Even at 0.1 km, the signal already loses about 80 dB, and the loss keeps growing, reaching around 120 dB by 10 km, but it slows down as distance increases. That's why regular 2.4 GHz Wi-Fi only really works over a few hundred meters; getting to kilometer-range distances would need directional antennas built for that job.

Assignment 02: 5G Core Network & Architecture

This assignment was about the move from EPC, the 4G core, to 5GC, the 5G core. The main idea is that 5GC uses an API-based design instead of a fixed structure, and that's what makes network slicing possible. I also wrote about Multi-access Edge Computing and why moving processing closer to the user cuts down on delay, how AI can help allocate network resources on the fly, and what the Network Repository Function actually does. It's basically a directory so different parts of the network can find each other.

What I learned

Network slicing was the idea that made the biggest impression on me. One physical 5G network can act like several different networks at once. One slice could be tuned for a self-driving car that can't afford any delay, and another for thousands of small IoT sensors that barely send any data. That's what actually sets 5G apart, not just faster speeds.

Module 3: AI and Machine Learning Fundamentals for Telecommunications

Activities: Assignment 03, AI/ML in RAN, SON & Open RAN | Laboratory 03, Network Traffic Prediction

Assignment 03: AI/ML Applications in RAN, SON, and Open RAN

This one connected AI to actual radio network operations. I wrote about how a network can adjust spectrum and power automatically based on traffic, and how it can catch problems early across the different radio components before they turn into outages. I also covered how Self-Organizing Networks fix themselves and flag maintenance needs ahead of time, how Open RAN splits hardware and software apart compared to older systems, and the two halves of the RAN Intelligent Controller. One half handles slower policy and training work, and the other makes fast radio decisions in under a second.

Connecting it to the lab

One question asked me to connect something from an earlier lab, normalizing user demand to a 0-1 scale before allocating resources, to real Open RAN scheduling. Writing that answer out helped me see that normalization isn't just a data-cleaning step; it's exactly what lets a scheduler treat a video stream and a small sensor reading fairly, based on need instead of raw numbers.

Laboratory 03: Network Traffic Prediction

For this one I used a Random Forest model to predict hourly network traffic, working off a year of realistic synthetic data that had daily, weekly, and seasonal patterns built into it. I added lag features (traffic from 1 hour ago and 24 hours ago) plus a 7-day rolling average, then trained on 80% of the data (8,570 rows total) and tested on the rest.

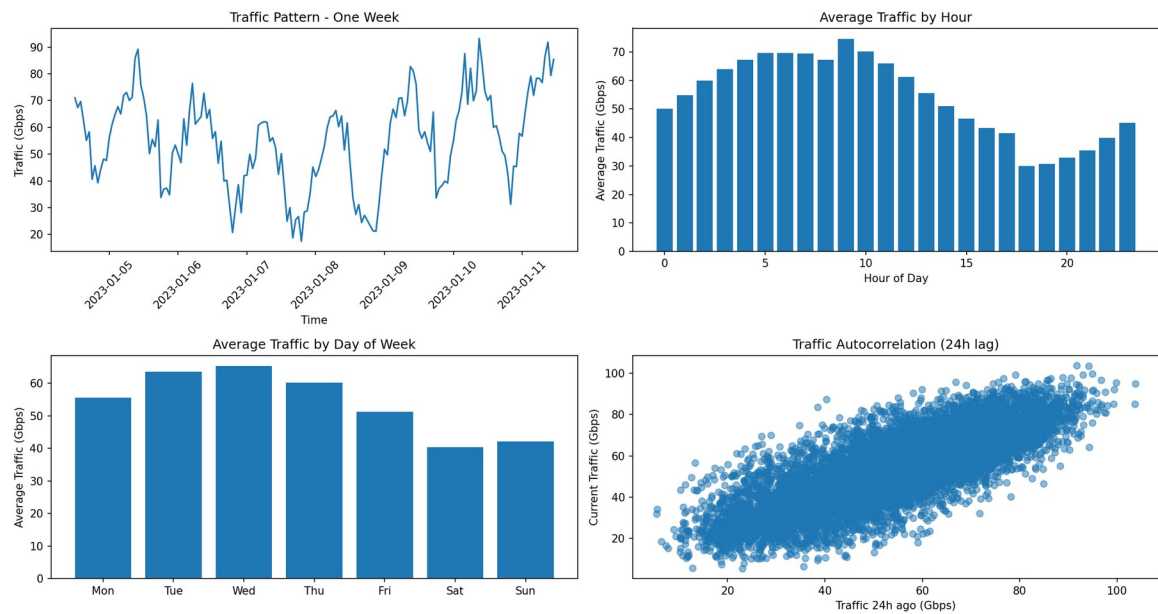


Figure 3.1: Traffic patterns by week, hour of day, day of week, and 24-hour lag

Metric	Training	Testing
MSE	17.59	34.64

Metric	Training	Testing
R^2	0.946	0.893

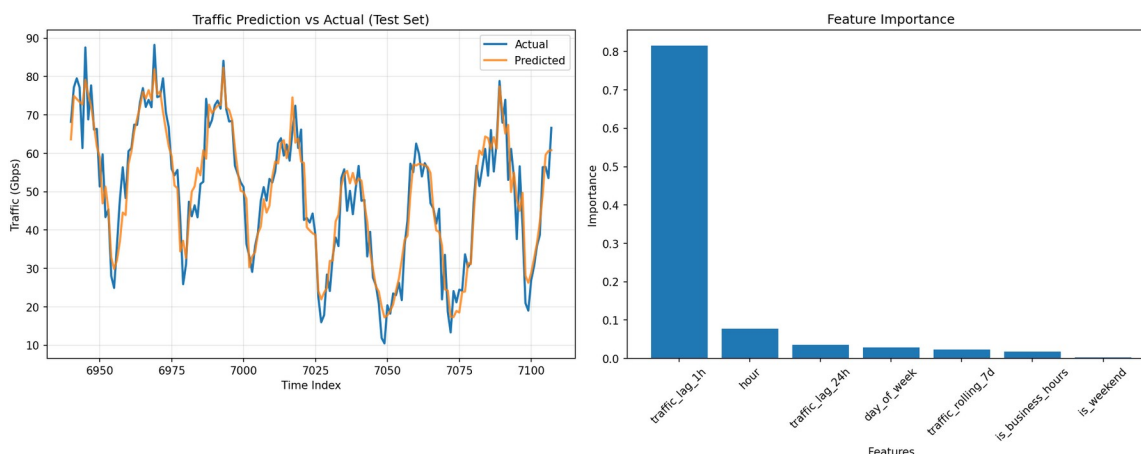


Figure 3.2: Predicted vs. actual traffic on the test set, with feature importance

The 1-hour lag feature was by far the most important one, at 81.6% of total importance. Traffic an hour ago is the strongest clue about traffic right now. Hour of day came in second, at 7.8%. The model held up well on new data too, with a test R^2 of 0.893, meaning it explained about 89% of the variation in traffic it hadn't seen before.

What I learned

This was the first time I really saw how much a model's accuracy depends on the features you build, not just the algorithm. The lag and rolling-average features I engineered mattered more here than whether I used a Random Forest or something else.

Module 6: AI for Network Optimization and Management

Activities: Laboratory 04, Traffic Prediction with Time-Series Forecasting (ARIMA & Linear Regression)

This lab was about using AI to forecast network traffic ahead of time, which is really the point of this module: predicting where the load is going so a network can manage its own resources instead of just reacting after the fact. I generated a full year of hourly traffic data, about 8,760 rows, with daily cycles, weekly cycles, a long-term trend, seasonal changes, and a few random traffic spikes mixed in. After exploring the data and breaking it into its trend, seasonal, and leftover-noise parts, I built and compared two models: an ARIMA model and a Linear Regression model built on engineered time-based features.

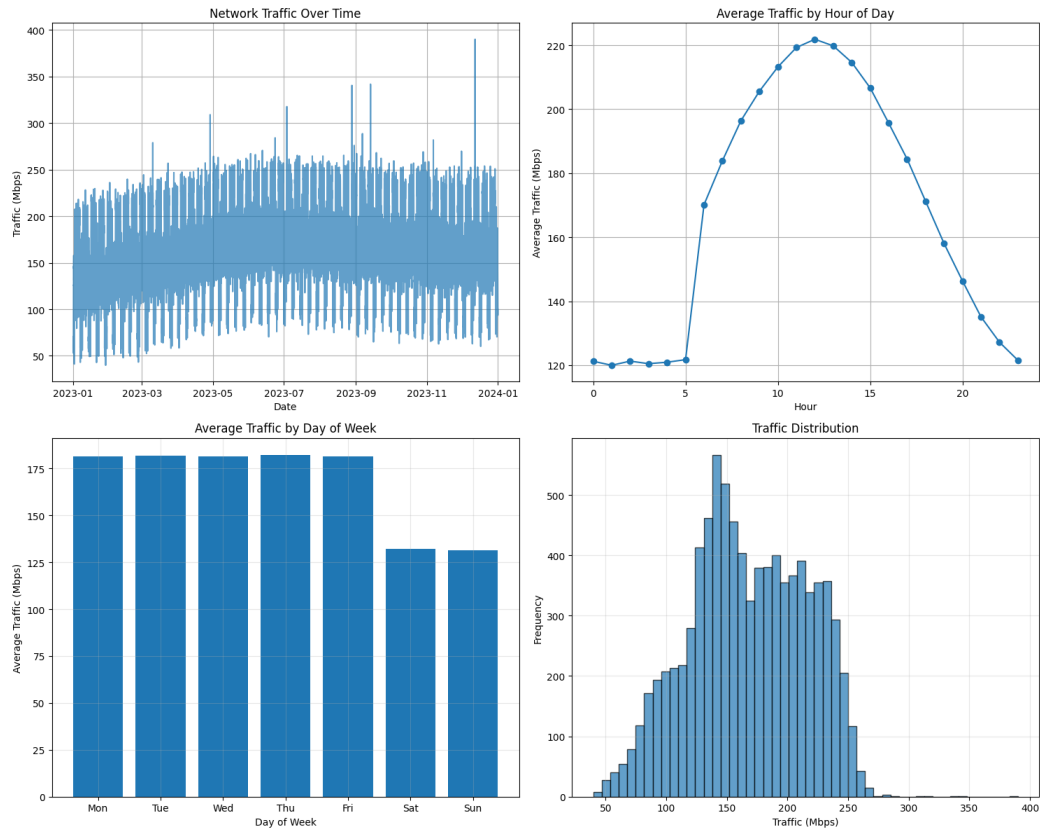


Figure 6.1: Traffic over time, by hour of day, by day of week, and overall distribution

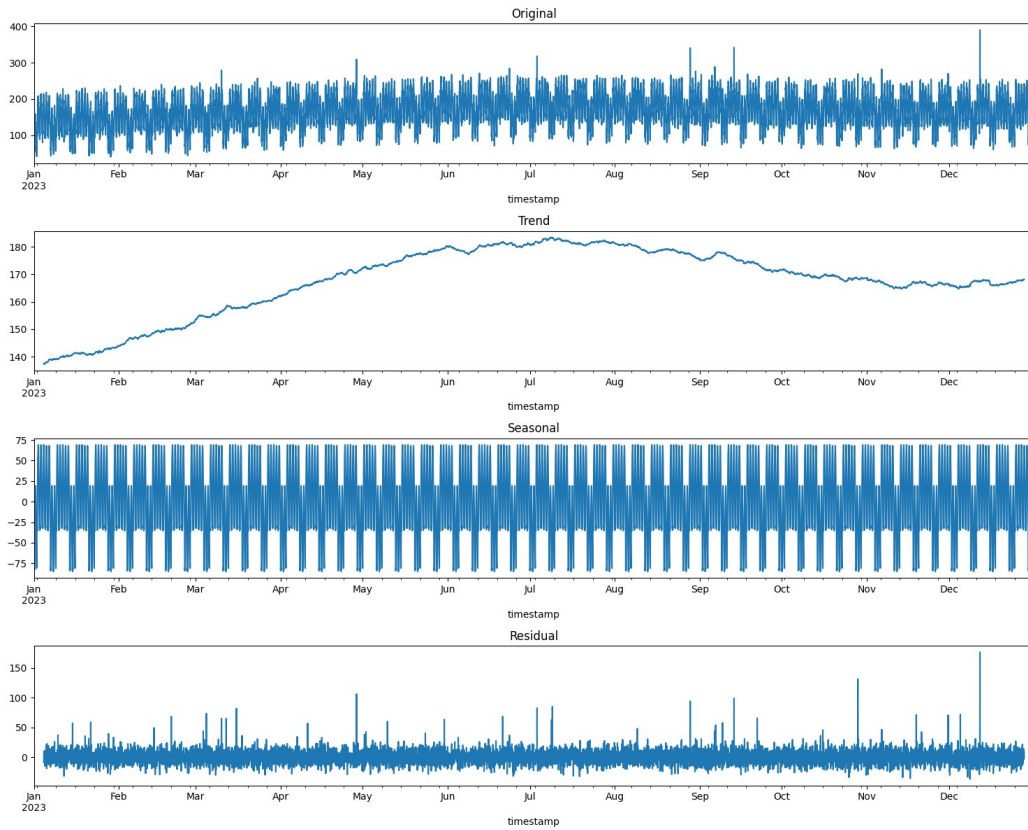


Figure 6.2: Breaking the traffic data into trend, weekly pattern, and leftover noise

An Augmented Dickey-Fuller test showed the data was stationary ($p < 0.001$), which is something you need to check before an ARIMA model will work properly.

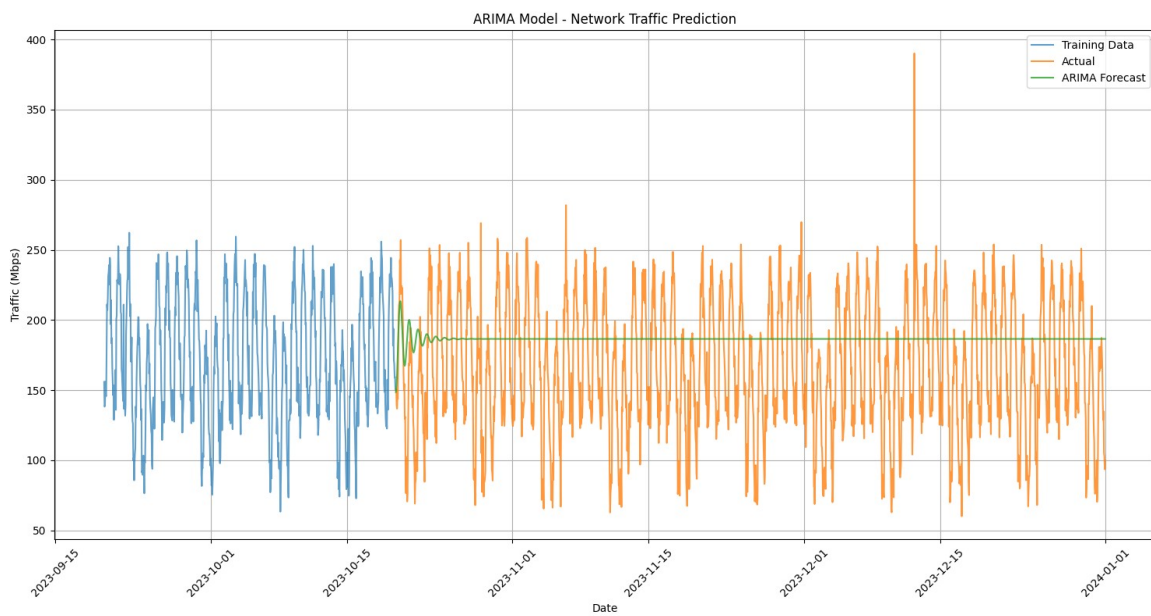


Figure 6.3: ARIMA(2,1,2) forecast against the actual test-set traffic

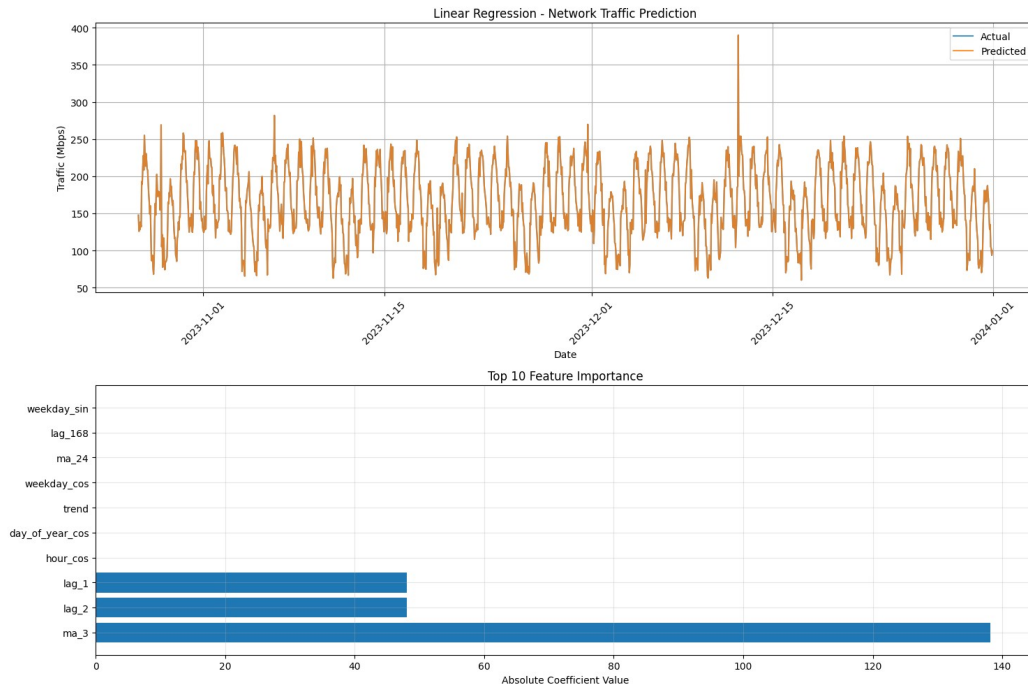


Figure 6.4: Linear regression predictions and its top 10 features

The Linear Regression model used hour, weekday, and day-of-year encoded as cycles, plus lag features going back from 1 hour up to a full week, along with rolling averages. It ended up scoring a near-perfect R^2 on the test set, mainly because the short lag and moving-average features already captured almost all of the pattern. Just like in Lab 03, the short-term lag features and the 3-hour moving average mattered most.

What I learned

Putting ARIMA and the engineered regression model side by side showed me the real difference between the two approaches. ARIMA picks up the pattern in the data on its own, using the series' own past values. The regression model needed me to build those same patterns by hand (lags, rolling averages, cyclical features) before it could do the same job.

Module 7: IoT and Edge Computing with AI

Activities: Laboratory 05, Edge Computing Simulation (Pruning, Quantization & Knowledge Distillation)

This lab was about running an AI model on a small IoT device at the network edge instead of in the cloud, which is really the core problem this module is about: how do you get AI to run on hardware that doesn't have much power or memory. I trained a baseline neural network to classify IoT sensor readings into 3 device states, using 20 input features. From there I compressed it three different ways. Pruning cut the network down to fewer layers and parameters. Quantization lowered the numerical precision using TFLite. Distillation trained a small "student" model to copy what the bigger "teacher" model was doing.

```
converter = tf.lite.TFLiteConverter.from_keras_model(baseline_model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
# INT8 quantization applied through the TFLite converter
```

Model	Accuracy	Inference Time (ms)	Size (KB)	Parameters
Baseline (Cloud)	94.15%	70.96	51.26	13,123
Pruned	93.20%	72.34	13.76	3,523
Distilled	91.30%	62.06	4.89	1,251
Quantized (INT8)	93.30%	0.50	21.58	13,123*

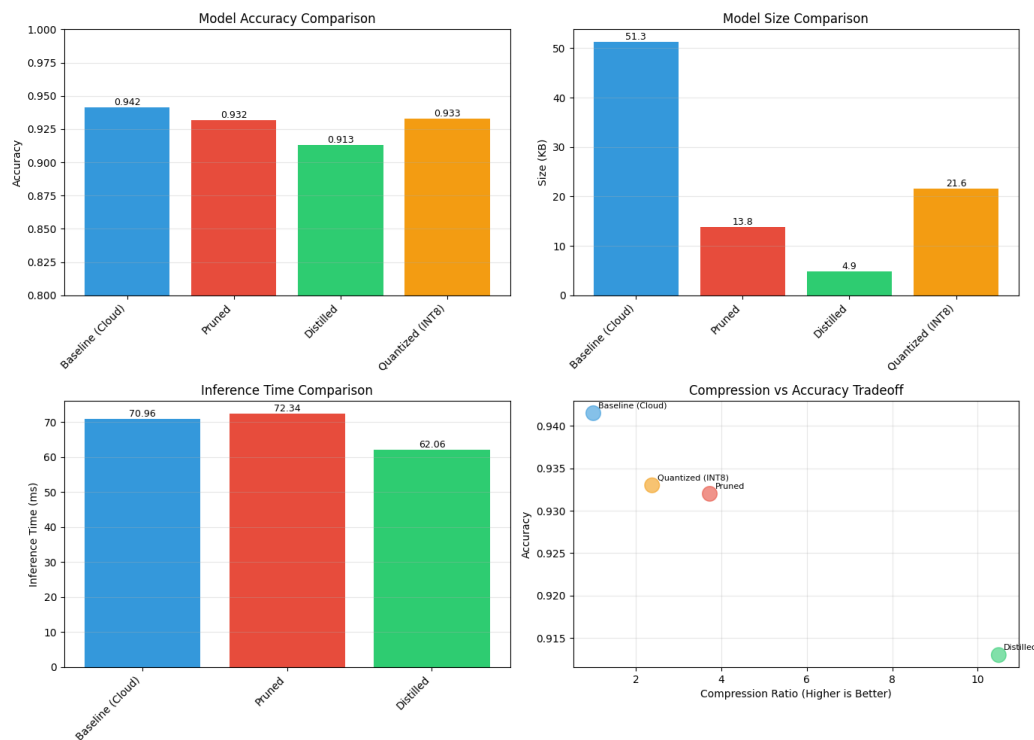


Figure 7.1: Accuracy, inference time, model size, and parameter count for all four versions

Quantization gave the biggest win on speed. Inference time dropped from about 71 ms to under 1 ms, while barely losing any accuracy. That makes it the obvious pick for something like a microcontroller. Distillation gave the smallest file size by far, under 5 KB, at the cost of about 3 points of accuracy, which fits better on a mid-size edge gateway. The lab's own recommendations lined up with this: quantized models for small sensors, pruned or distilled models for gateways, and the full baseline model for edge servers doing heavier, more critical work like self-driving cars.

What I learned

What surprised me most was how much faster quantization made the model without really hurting accuracy, almost 140 times faster for well under a 1% drop. It changed how I think about edge deployment. I went in assuming smaller model size and faster inference always go hand in hand, but this lab showed they don't move together the way I expected.

Module 13: Ethical AI and Future Trends

Activities: Assignment 04, Ethical AI (30-question written assignment)

This assignment was split into three parts. The first part was more conceptual, covering things like fairness, transparency, and privacy, plus a look at frameworks like the EU AI Act and how different organizations approach AI regulation. The second part worked through real scenarios, like a biased hiring tool or the self-driving car trolley problem. The last part was more personal, asking things like whether AI can ever be fully unbiased, and what I plan to actually do about all of this once I'm working in the field.

A few things that stood out

Algorithmic bias in practice

I used Amazon's old hiring tool as an example of algorithmic bias. It started rejecting resumes with the word "women's" in them because it had learned from years of resumes mostly submitted by men. Writing that example out made it clearer to me that bias usually comes from old data reflecting old patterns, not from someone deliberately programming discrimination.

Regulation based on risk level

Working through how the EU AI Act sorts AI systems by risk level, banning things like government social scoring outright while requiring heavy testing and human oversight for high-risk uses like hiring or policing, gave me a real example of what "regulation that matches the risk" looks like in practice, not just as an idea.

My own role going forward

The last question asked how I'd actually practice ethical AI in my own career. My answer was to check for bias the whole way through a project, not just before launch, to be upfront with the people a system actually affects, and to keep learning about new rules and best practices instead of treating it as something I only need to learn once.

What I learned

This assignment wasn't really about learning new technical material. It was more about practicing how to think through tradeoffs that don't have a clean answer, like where to draw the line between regulation and innovation, or who's actually responsible when an AI system gets something wrong. Out of everything in this course, this is the part I expect to use the most once I'm working, since these kinds of judgment calls come up before any radio or ML technique does.

Overall Reflection

Looking back, this class covered a lot of ground: signals and noise, network architecture, AI models, and the ethics of actually using them. If there's one thing that ties it all together, it's that almost everything we did came back to the same basic question: how do you make the most of a limited resource when demand keeps shifting around? Sometimes that resource was bandwidth on a radio channel, sometimes it was a few kilobytes of memory on a small sensor, but the underlying problem kept showing up in different forms.

The labs built on real or realistic data, like the Wireshark capture and the traffic data with its daily and weekly cycles, taught me more than the purely theoretical parts, because they forced me to deal with the messy stuff (noise, missing values from lag features, non-stationary data) that a textbook usually skips past. Going forward, the two things I want to carry out of this class are the ability to reason about a network at the architecture level, and the ability to actually build and test a model against it, all while keeping the ethics questions from the Ethical AI module in mind the whole time instead of as an afterthought.

Citations & Sources

- Course materials, lecture slides, and lab instructions from Professor Tawanda Chiyangwa, ITAI-4370, Houston Community College.
- Wireshark, open-source packet analysis software, [wireshark.org](https://www.wireshark.org).
- Python libraries used throughout: NumPy, SciPy, pandas, matplotlib, seaborn, scikit-learn, statsmodels, and TensorFlow/Keras.
- 3GPP and O-RAN Alliance specifications, referenced for 5G Core (5GC), Open RAN interfaces (E2, A1, O1), and RIC architecture.
- UNESCO Recommendation on the Ethics of Artificial Intelligence; OECD Principles on Artificial Intelligence; EU Artificial Intelligence Act, referenced in Module 13 / Assignment 04.
- All lab code, simulations, and written answers in this portfolio are my own work, completed individually for ITAI-4370.